Practice 4. Palindromes and Balance

[CSE2010] Data Structures

Department of Data Science

Overview

- Problem 1. check if a given string is a **palindrome**
- Problem 2. given a string including round, curly, and square opening and closing brackets, return whether the brackets are **balanced** (well-formed).

Problem 1. Palindrome

- $Palindromes = \{\boldsymbol{w} \mid \boldsymbol{w}\ reads\ the\ same\ left\ to\ right\ as\ right\ to\ left\}$
  - mom
  - level
  - tenet
- Check if a given string is a **palindrome**
  - Example 1
    - "level“ is a palindrome
  - Example 2
    - “coffee“ is not a palindrome

Problem 2. Balance

- Given a string including round, curly, and square opening and closing brackets, return whether the brackets are **balanced**.
- A string is called **balanced** if
  1. the number of opening brackets is the same as the number of closing brackets, and
  2. An opening bracket comes earlier than its corresponding closing bracket.

- **Example 1**
  - Input: ‘{{}}{}’
  - Output: True
- **Example 2**
  - Input: ‘{{{}}}’
  - Output: True
- **Example 3**
  - Input: ‘{{ }{’
  - Output: False
- **Example 4**
  - Input: ‘{{ }’
  - Output: False

Implementation

- Function isPalindrome
- Function balance

```
// Return true if str is a palindrome; false
otherwise
bool isPalindrome(string& str) {

}
// Return true if brackets are balanced in str;
false otherwise
bool balance(string& str) {

}
```

```
# Return True if str is a palindrome; False
otherwise
def isPalindrome(string):
  pass

# Return True if brackets are balanced in str;
False otherwise
def balance(string):
  pass
```

Each Line Represents an Operation in Input File

- **P<space>[string]**
  - If [string] is a palindrome, write “T” into the output file.
  - Otherwise, write “F” into the output file.
- **B<space>[string]**
  - If all the brackets are balance in [string] , write “T” into the output file.
  - Otherwise, write “F” into the output file.
- Each line of output file represents to the result of the corresponding line of the input file.
- Input file

```
P level
P hello
B ([a])[b]({})
B ([)]
```

- Output file

```
T
F
T
F
```

Running Example

C/C++

```
[hjkim@localhost stacks]$ make
g++ -std=c++11 -o assignment1 stack.cpp
[hjkim@localhost stacks]$ make run
./assignment1 input.txt output.txt
[hjkim@localhost stacks]$ make clean
rm assignment1
[hjkim@localhost stacks]$
```

Python

```
[hjkim@localhost stacks]$ make
python3 stack.py input.txt output.txt
[hjkim@localhost stacks]$
```

Running Example

- [Optional] “make run” or “make” will run your predefined commands.

- makefile

```
all: practice3

practice3: queue.cpp
    g++ -std=c++11 -o practice3
queue.cpp

run: practice3
    ./practice3 queue_input.txt
c++_queue_output.txt

clean: practice3
    rm practice3
```

```
[hjkim@localhost queues]$ make
g++ -std=c++11 -o practice3 queue.cpp
[hjkim@localhost queues]$ make run
./practice3 queue_input.txt c++_queue_output.txt
[hjkim@localhost queues]$ make clean
rm practice3
```

Evaluation

- Write following comments in your source code
  - **Input, output, and description** of functions
  - **Description** on main variables
  - **Time complexity** of your algorithms
- Results evaluated by test cases

Submission Guideline

- Submission: a compressed file that contains
  - **source code**
  - **your own input file**
  - **output file**
    - produced by running your source code on the above input file
- Where: Assignments in LMS
- Deadline: **23:59, March 31 (Sunday)**

Make

- Build tools that have been around for a long time on Unix operating systems
  - Rules for how to compile and link the source to create an executable
- When "**make**" is run, this automatically finds "**makefile**" (or "**Makefile**") in that directory and run it.
- Writing a makefile
  - target: the name of a file to create (such as object or executable file)
  - prerequisites: a list of targets needed to create the target
  - command(s): each step to create a target. <tab> must be placed before the command.

```
target: prerequisites
<tab>command1
<tab>command2
```

Example: Writing a makefile

- Write a makefile

(Shell – working directory)
$ vi makefile

```
hello: main.c
        g++ -o hello main.c
run: hello
        ./hello input.txt output.txt
clean:
        rm hello
```

Example: Running a makefile

- Run the first target command

(Shell – working directory)
$ make

- Run the specified target

(Shell – working directory)
$ make run

- Run the specified target to remove an executable file

(Shell – working directory)
$ make clean

Hints for Python Users

- In Python, implement a stack with **list**
  - Python list is implemented with a dynamic array.
  - A stack implemented by a list takes O(1) time to push and pop.
    - Adding an element to the right end and deleting an element from the right end take O(1).
- In Python, implement a queue with **collections.deque**
  - A queue implemented by Python list takes O(n) time to enqueue and dequeue (where n is the number of elements in the queue)
  - Fortunately, Python provides a built-in library that implements a queue.
    - **from collections import deque**

from collections import deque

```
from collections import deque


q = deque()         # Create an empty queue
for i in range(5):
    q.append(i)     # Add 1,2,…,5 to a queue


print(q.popleft()) # Delete & output the 1st elem
```

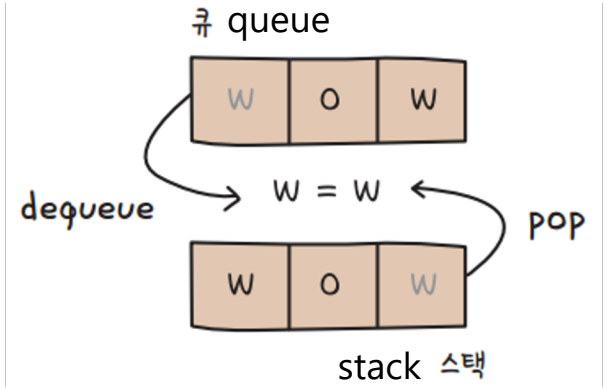
Hints for C++ Users

- In C++, use the stack and queue implementation provided by STL libraries.
- **#include <stack>**
- **#include <queue>**

Hints for Recognizing Palindromes

Hints

- Suppose that you put every character into a queue and a stack, and then delete elements from the queue and the stack one by one.
- If each character deleted from a queue is the same as each character deleted from a stack, then this string is a palindrome



Hints for Recognizing Palindromes

Hints

- In fact, there is another solution that does not need a queue and a stack.